

# Pagination

- Pagination is the process of dividing a large body of content into smaller, manageable pages or sections for easier viewing and navigation, both in digital and print media.
- In digital contexts like websites, it typically involves displaying a limited number of items per page, with controls like "Next," "Previous," and page numbers allowing users to move between sets of content.
- This user interface pattern makes large datasets and long documents less overwhelming and improves loading times and user experience.

# Spring boot pagination

- In Spring Boot, the most common and robust way to implement pagination is by using Spring Data JPA's **Pageable and Page interfaces**.
- These provide an efficient and easy-to-use abstraction for handling large datasets by fetching data in smaller, manageable chunks.
- **Pageable**  
An interface used to define pagination information, including the page number (zero-indexed) and the number of records per page (page size). It can also encapsulate sorting criteria.
- **Page**  
An interface representing a single page of data returned from a paginated query. It contains the list of elements for the current page, along with metadata such as the total number of pages, total elements, and whether it's the first or last page.

# PageRequest and Sort

- PageRequest
  - A concrete implementation of Pageable that is commonly used to create Pageable instances.
  - create a PageRequest using PageRequest.of(pageNumber, pageSize, sortCriteria).
- Sort
  - An interface used to define sorting logic, specifying fields to sort by and the sort direction (ascending or descending).
  - Sort.by(sortDirection, propertyName) allows specifying the sorting order (ascending or descending) and the field(s) to sort by.
- PagingAndSortingRepository (or JpaRepository):
  - These interfaces, extended by your custom repositories, provide methods that accept Pageable as a parameter, allowing to fetch paginated and sorted data directly from the database.

# Example

@Repository

```
public interface ProductRepository extends JpaRepository<Product, Long> {  
}
```

```
class Product {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

```
    private double price;
```

```
}
```

# Service file

```
@Service
```

```
public class ProductService {
```

```
    @Autowired
```

```
    private ProductRepository productRepository;
```

```
    public Page<Product> getAllProduct(Pageable pageable){
```

```
        return productRepository.findAll(pageable);
```

```
    }
```

```
    public Page<Product> getPaginatedProduct(int page, int size, Sort sortBy) {
```

```
        Pageable pageable = PageRequest.of(page, size, sortBy);
```

```
        return productRepository.findAll(pageable);
```

```
    }
```

```
}
```

# controller

```
@RestController
@RequestMapping("/products")
public class ProductController {
    @Autowired
    private ProductService productService;

    @GetMapping
    public Page<Product>
    getProduct(Pageable page){
        return
        productService.getAllProduct(page);
    }
}
```

<http://localhost:8080/products?page=0&size=10>

<http://localhost:8080/products?page=1&size=10>

<http://localhost:8080/products?page=2&size=10>

<http://localhost:8080/products?page=0&size=11&sort=id,desc>

# main

## @SpringBootApplication

```
public class DemoApplication implements  
CommandLineRunner {
```

```
public static void main(String[] args) {  
    SpringApplication.run(DemoApplication.class, args);  
}
```

@Autowired

```
ProductRepository productRepository;
```

@Autowired

```
private ProductService productService;
```

## @Override

```
public void run(String... args) throws Exception {
```

```
var list1 = Arrays.asList(
    new Product[]{
        Product.builder().name("biscuit").price(40).build(),
        Product.builder().name("biscuit").price(40).build(),
        Product.builder().name("biscuit").price(40).build(),
        Product.builder().name("biscuit").price(40).build(),
```

```
productRepository.saveAll(list1)
```

```
productRepository.saveAll(list1)
```

```
productRepository.saveAll(list1)
```

```
Pageable pageable = PageRequest.of(1, 10);
```

```
System.out.println("using pagination ");
```

```
Page<Product> myEntitiesPage =
productService.getAllProduct(pageable);
```

```
myEntitiesPage.getContent().forEach(System.out::println);
```

```
Page<Product> page =
productService.getPaginatedProduct(0, 2,
Sort.by("name").ascending());
```

```
page.getContent().forEach(System.out::println);
```

# Pagination in angular

- Implementing pagination between a Spring Boot backend and an Angular frontend involves both server-side and client-side logic.
- Service: Create an Angular service to make HTTP requests to your Spring Boot API. Pass the page and size parameters.
- ng g service

# Angular http params

- Angular's `HttpParams` class, found in `@angular/common/http`.
  - It is used to manage and send query parameters with HTTP requests, particularly with `HttpClient`.
  - These parameters are appended to the URL as key-value pairs.
  - Key characteristics and usage:
    - Immutability: `HttpParams` instances are immutable.
    - Methods like `set()` or `append()` do not modify the existing instance but instead return a new `HttpParams` instance with the changes applied.
    - This means you must reassign the variable when modifying parameters.
- ```
import { HttpParams } from '@angular/common/http';
```

```
let params = new HttpParams();  
params = params.set('filter', 'all'); // Reassign the new instance  
params = params.append('sort', 'name,asc'); // Reassign again
```

# service

```
@Injectable({
  providedIn: 'root'
})
export class ProductService {
  // private baseUrl =
  'http://localhost:8080/products?page=0&size=5&sort=id,asc';
  private baseUrl = 'http://localhost:8080/products';

  constructor(private http: HttpClient) {}

  getItems(page: number, size: number): Observable<any> {
    let params = new HttpParams()
      .set('page', page.toString())
      .set('size', size.toString());
    return this.http.get<any>(this.baseUrl, { params });
  }
}
```

```
export class ProductComponent {
  items: any[] = [];
  currentPage = 0;
  pageSize = 5;
  totalPages = 0;
  totalElements = 0;
  constructor(private pservice: ProductService) {}
  ngOnInit(): void {
    this.loadProducts();
  }
  loadProducts(): void {
    this.pservice.getItems(this.currentPage,
      this.pageSize).subscribe(data => {
      this.items = data.content;
      this.totalPages = data.totalPages;
      this.totalElements = data.totalElements;
    });
  }
  onPageChange(page: number): void {
    this.currentPage = page;
    this.loadProducts();
  }
}
```

# Component template

```
<table class="table table-bordered p-5 m-5">
  <tr>
    <th>name</th>
    <th>price</th>
  </tr>
  <tr *ngFor="let item of items">
    <td> {{ item.name }}</td>
    <td> {{ item.price }}</td>
  </tr>
</table>
```

```
<button (click)="onPageChange(currentPage - 1)"
[disabled]="currentPage === 0">Previous</button>
<span>Page {{ currentPage + 1 }} of {{ totalPages }}</span>
<button (click)="onPageChange(currentPage + 1)"
[disabled]="currentPage === totalPages - 1">Next</button>
```

## app.config.ts

```
providers: [provideZoneChangeDetection({ eventCoalescing:
true }),
provideRouter(routes),
provideHttpClient(), // Provide HttpClient
]
```